

3차 과제에서 만든 야구 타자 성적 관리 시스템을 4차 과제 조건인 배열과 포인터 중심 구조로 어떻게 확장하면 좋을까?

승우님, 3차 과제에서 만든 야구 타자 성적 관리 시스템은 이미 `while(1)` 메뉴 구조와 함수 분리가 되어 있으므로, 4차 과제에서는 이 구조를 유지하면서 단일 타자 관리 방식에서 여러 타자 관리 방식으로 확장하면 좋아요.

가장 큰 변화는 기존에 한 명의 타자 정보만 저장하던 변수를 배열로 바꾸는 것입니다. 예를 들어 3차에서는 `initial`, `game`, `at_bat`, `hit`, `home_run` 처럼 하나의 값만 저장했다면, 4차에서는 최대 5명의 타자를 저장할 수 있도록 `initial[5]`, `game[5]`, `at_bat[5]`, `hit[5]`, `home_run[5]` 같은 1차원 배열로 확장할 수 있습니다.

또한 기존 메뉴 구조는 그대로 유지하되, 기능 이름을 "타자 정보 입력"에서 "여러 타자 정보 입력", "현재 타율 계산"에서 "전체 타율 계산", "타자 성적 출력"에서 "전체 타자 성적 출력"처럼 바꾸면 자연스럽습니다. 사용자는 메뉴를 통해 여러 명의 타자를 입력하고, 전체 타자의 타율을 계산한 뒤, 각 타자의 성적과 등급을 확인할 수 있습니다.

4차 과제에서는 배열을 단순히 선언하는 것뿐만 아니라 순회 방식도 중요합니다. 성적 출력 기능에서는 `arr[i]` 와 같은 인덱스 방식을 사용해 여러 타자의 정보를 출력하고, 타율 계산 기능에서는 포인터 변수에 배열의 시작 주소를 저장한 뒤 `*(ptr + i)` 형태로 배열 요소에 접근하면 포인터 연산 조건도 충족할 수 있습니다.

마지막으로 배열을 함수의 매개변수로 전달해야 하므로, 예를 들어 `calculate_all_stats(hit, at_bat, last_avg, current_avg, avg_diff, player_count)` 처럼 배열과 타자 수를 함수에 넘겨서 전체 타율을 계산하도록 만들면 좋습니다. 이렇게 하면 3차에서 만든 함수 중심 구조를 유지하면서도, 4차 과제의 핵심인 배열과 포인터 활용까지 자연스럽게 포함할 수 있습니다.

📄 🌟 📌 ↕ 🔄 ...

기존에는 한 명의 타자 정보만 저장했는데, 최대 5명의 타자 정보를 관리하려면 어떤 데이터를 배열로 바꾸면 좋을까?

기존 3차 과제에서는 한 명의 타자 정보만 관리했기 때문에 각 데이터를 하나의 변수에 저장했습니다. 하지만 4차 과제에서는 최대 5명의 타자 정보를 저장해야 하므로, 여러 개의 데이터를 저장할 수 있는 배열로 변경하는 것이 좋습니다.

예를 들어 타자의 이니셜은 문자 하나만 저장하던 `initial` 변수를 `initial[5]` 로 변경하고, 경기 수는 `game[5]`, 타수는 `at_bat[5]`, 안타 수는 `hit[5]`, 홈런 수는 `home_run[5]` 처럼 배열로 선언할 수 있습니다. 또한 지난 시즌 타율, 현재 타율, 타율 차이도 각각 `last_avg[5]`, `current_avg[5]`, `avg_diff[5]` 배열로 관리할 수 있습니다.

이렇게 하면 배열의 인덱스가 각 타자를 구분하는 역할을 하게 됩니다. 예를 들어 `initial[0]`, `game[0]`, `hit[0]` 은 1번 타자의 정보이고, `initial[1]`, `game[1]`, `hit[1]` 은 2번 타자의 정보가 됩니다. 즉 같은 인덱스를 사용하는 데이터들이 하나의 타자 정보를 구성하게 됩니다.

배열로 변경하면 `for` 문을 사용하여 여러 타자의 정보를 반복적으로 입력받거나 출력할 수 있다는 장점도 있습니다. 따라서 4차 과제에서는 타자의 이니셜, 경기 수, 타수, 안타 수, 홈런 수, 지난 시즌 타율, 현재 타율, 타율 차이와 같이 여러 명의 타자에 대해 저장해야 하는 데이터를 모두 배열로 변경하는 것이 가장 적절합니다.

📄 👍 🔒 ↕ ↻ ...

4차 과제에서 인덱스 방식 배열 순회와 포인터 연산 방식 배열 순회를 각각 어디에 적용하면 좋을까?

승우님, 4차 과제에서는 두 방식이 모두 들어가야 하니까 역할을 나누는 게 좋아요.

인덱스 방식 배열 순회는 여러 타자의 성적을 출력할 때 적용하는 것이 가장 자연스럽습니다. 예를 들어 `print_all_players()` 함수에서 `for` 문을 사용해 `initial_arr[i]`, `game_arr[i]`, `hit_arr[i]` 처럼 각 배열의 `i` 번째 값을 출력하면 됩니다.

```
<> C
for (i = 0; i < count; i++)
{
    printf("타자 미니셜\t: %c\n", initial_arr[i]);
    printf("경기 수\t\t: %d\n", game_arr[i]);
    printf("안타 수\t\t: %d\n", hit_arr[i]);
}
```

반면 포인터 연산 방식 배열 순회는 전체 타율을 계산할 때 적용하는 것이 좋습니다. 예를 들어 `calculate_all_stats()` 함수에서 배열의 시작 주소를 포인터 변수에 저장한 뒤, `*(ptr + i)` 방식으로 안타 수와 타수에 접근하여 현재 타율을 계산하면 됩니다.

```
<> C
hit_ptr = hit_arr;
at_bat_ptr = at_bat_arr;

for (i = 0; i < count; i++)
{
    *(current_avg_ptr + i) = calculate_average(*(hit_ptr + i), *(at_bat_ptr + i));
}
```

이렇게 역할을 나누면 README에 적은 내용처럼 출력 기능에서는 인덱스 방식, 계산 기능에서는 포인터 연산 방식을 사용하게 되어 4차 과제 조건을 명확하게 충족할 수 있습니다.

📄 🍌 🔒 ⬆ ↺ ...

여러 타자 정보를 출력하거나 분석할 때 배열을 함수의 매개변수로 전달하려면 어떤 구조로 작성하면 좋을까?

배열에 저장된 여러 타자의 정보를 출력하거나 분석할 때는 배열을 함수의 매개변수로 전달하여 처리하는 구조가 가장 효율적입니다. 만약 전역변수만 사용하면 함수 간의 의존성이 커지고 재사용성이 떨어질 수 있으므로, 필요한 배열과 저장된 타자 수를 함께 전달하는 방식이 좋습니다.

예를 들어 전체 타자 성적을 출력하는 함수는 다음과 같이 배열을 매개변수로 받을 수 있습니다.

↗ C



```
void print_all_players(  
    char initial_arr[],  
    int game_arr[],  
    int at_bat_arr[],  
    int hit_arr[],  
    int home_run_arr[],  
    float last_avg_arr[],  
    float current_avg_arr[],  
    float avg_diff_arr[],  
    int count)
```

이렇게 하면 함수 내부에서는 `for` 문을 사용하여 배열을 순회하면서 각 타자의 정보를 출력할 수 있습니다.

↗ C



```
for (i = 0; i < count; i++)  
{  
    printf("타자 미니셜 : %c\n", initial_arr[i]);  
    printf("경기 수 : %d\n", game_arr[i]);  
}
```

또한 전체 타자를 분석하는 함수도 비슷한 방식으로 작성할 수 있습니다.

↗ C



```
void analyze_all_players(  
    char initial_arr[],  
    float current_avg_arr[],  
    int home_run_arr[],  
    int count)
```



그리고 함수를 호출할 때는 실제 배열과 저장된 타자 수를 전달합니다.

↗ C

```
print_all_players(  
    initial,  
    game,  
    at_bat,  
    hit,  
    home_run,  
    last_avg,  
    current_avg,  
    avg_diff,  
    player_count);
```

이러한 구조를 사용하면 배열을 함수로 전달하는 4차 과제 조건을 만족할 수 있고, 여러 타자의 데이터를 함수 별로 체계적으로 처리할 수 있습니다. 또한 배열과 저장 개수(`player_count`)를 함께 전달하기 때문에 입력된 타자 수만큼만 반복문을 수행하도록 구현할 수 있다는 장점도 있습니다.

📄 👍 🗨 📌 🔄 ...

여러 명의 타자 정보를 출력할 때 같은 코드를 반복해서 작성하지 않고 for문과 배열을 활용하면 어떤 장점이 있을까?

여러 명의 타자 정보를 출력할 때 같은 코드를 반복해서 작성하지 않고 for 문과 배열을 활용하면 코드의 길이를 크게 줄일 수 있고, 유지보수도 훨씬 쉬워집니다.

예를 들어 타자가 5명이라면 배열을 사용하지 않을 경우 각 타자마다 출력문을 따로 작성해야 합니다. 하지만 배열과 for 문을 사용하면 하나의 반복문으로 모든 타자의 정보를 출력할 수 있습니다.

↗ C



```
for (i = 0; i < count; i++)
{
    printf("타자 이니셜 : %c\n", initial[i]);
    printf("경기 수 : %d\n", game[i]);
    printf("안타 수 : %d\n", hit[i]);
}
```

이처럼 반복문을 사용하면 타자 수가 늘어나더라도 코드를 추가로 작성할 필요가 없습니다. 또한 출력 형식을 수정해야 할 경우에도 반복문 내부만 수정하면 되므로 관리가 편리합니다.

따라서 4차 과제에서는 여러 명의 타자 정보를 처리할 때 배열과 for 문을 함께 사용하여 코드의 중복을 줄이고, 프로그램을 더 효율적으로 구성할 수 있습니다. 이는 배열 순회 조건을 충족하는 동시에 코드의 가독성과 확장성도 높여주는 장점이 있습니다.

📄 👍 🔒 ⬆️ ↺ ...

배열과 포인터를 함께 사용할 때 `arr[i]` 방식과 `*(ptr + i)` 방식이 헷갈리는데, 두 방식은 어떤 관계이고 언제 사용하는 것이 좋을까?

배열과 포인터를 함께 사용할 때 `arr[i]` 방식과 `*(ptr + i)` 방식은 서로 다른 것처럼 보이지만, 실제로는 같은 위치의 데이터를 가리키는 방법입니다.

예를 들어 다음과 같은 배열이 있다고 가정해보겠습니다.

```
↗ C
int hit[5] = {165, 144, 118, 88, 70};
```

그리고 포인터가 배열의 시작 주소를 저장하고 있다면,

```
↗ C
int *ptr;
ptr = hit;
```

아래 두 코드는 같은 값을 의미합니다.

```
↗ C
hit[2]
```

```
↗ C
*(ptr + 2)
```

둘 다 배열의 세 번째 요소인 118에 접근하는 코드입니다. 즉, 배열 인덱스 방식은 내부적으로 포인터 연산을 이용하여 동작한다고 볼 수 있습니다.

4차 과제에서는 두 방식을 각각 다른 용도로 사용하는 것이 좋습니다. 성적 출력처럼 가독성이 중요한 부분에서는 `arr[i]` 방식이 직관적이므로 사용하기 편합니다.

↗ C

```
printf("안타 수 : %d\n", hit[i]);
```



반면 포인터 학습 목적이 있는 계산 함수에서는 `*(ptr + i)` 방식을 사용하는 것이 좋습니다.

↗ C

```
*(current_avg_ptr + i)  
    = calculate_average(*(hit_ptr + i),  
                        *(at_bat_ptr + i));
```



이렇게 하면 포인터 연산을 이용한 배열 순회 조건을 충족할 수 있습니다.

정리하면 `arr[i]` 와 `*(ptr + i)` 는 같은 데이터를 가리키는 두 가지 표현 방법이며, 출력이나 일반적인 데이터 접근에는 `arr[i]` 를 사용하는 것이 읽기 쉽고, 포인터 활용을 보여주어야 하는 계산이나 순회 기능에서는 `*(ptr + i)` 를 사용하는 것이 적절합니다.